

The sample application code shown below has a button captioned 'Send System Intent'; on clicking it, the application tries to send the 'ACTION_BATTERY_CHANGED' intent. However, what happens is that as soon as it is clicked, you will see the 'Unfortunately the app has stopped' dialogue (Figure 1). This is because a security exception happens when your application sends a system intent:

```
public class IntentTestActivity extends Activity {
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        /** When we click this Button, it tries to send System
        Intent */
        Button intentBtn = (Button)findViewById(R.id.intent);
        intentBtn.setOnClickListener(new Button.OnClickListener()
        {
            public void onClick(View v) {
                Intent myIntent=new Intent();
                myIntent.setAction(Intent.ACTION_BATTERY_
                CHANGED);
                getBaseContext().sendBroadcast(myIntent);
            }
        });
    }
}
```

A part of the stack trace of the exception message is as follows:

```
E/AndroidRuntime(18585): FATAL EXCEPTION: main
E/AndroidRuntime(18585): java.lang.SecurityException: Permission
Denial: not allowed to send broadcast android.intent.action.
BATTERY_CHANGED from pid=18585, uid=10064
```

As a result of this, the Activity Manager kills the application. Thus, the Android system makes sure that no application can send a system intent. You can try this with the help of the source code provided at http://www.linuxforu.com/article_source_code/jan13/intents_android.zip and by using an Android emulator:

```
W/ActivityManager( 263): Force finishing activity com.example.
intenttest/.IntentTestActivity
I/WindowManager( 263): WIN DEATH: Window{2100b9b8 com.
example.intenttest/com.example.intenttest.IntentTestActivity
paused=false}
I/ActivityManager( 263): Process com.example.intenttest (pid
18585) has died.
```

Creating system intents

However, if you are at an OEM/ODM, creating your own

system-level intent is an easy two-step process. First, add the intent name in `frameworks/base/core/java/android/content/intent.java`, as shown below:

```
/**
 * Broadcast Action: Have the device reboot. This is only
 for use by
 * system code.
 *
 * <p class="note">This is a protected intent that can only
 be sent
 * by the system.
 */
@SdkConstant(SdkConstantType.BROADCAST_INTENT_ACTION)
public static final String ACTION_REBOOT = "android.intent.
action.REBOOT";
```

Next, add the intent string inside the list of protected intents in `frameworks/base/core/res/framework/AndroidManifest.xml`. A part of this file looks like what follows:

```
<protected-broadcast android:name="android.intent.action.NEW_
OUTGOING_CALL" />
<protected-broadcast android:name="android.intent.action.REBOOT"
/>
<protected-broadcast android:name="android.intent.action.
MYACTION" />
```

But if you are really adding a system intent that does not exist in Android, you might want to up-stream your change to Google, and walk into the hall of fame!

Android CTS

Google provides an automated test suite that performs thorough regression/performance testing on various Android devices. This framework is called the Compatibility Test Suite (CTS). This exhaustive testing also includes verifying the security of system intents. The code that does this test resides in `cts/tests/tests/permission2/src/android/permission2/cts/ProtectedBroadcastsTest.java`. So, to make Android CTS test your system intent, add it to the list of intents (actually, it's an array of Strings named BROADCASTS) mentioned in this file. 

References

- [1] <http://developer.android.com/guide/components/intents-filters.html>
- [2] Android Source repository

By: Durgadoss R

The author is a kernel programmer, and spends his spare time hacking Android. You can reach him at r.durgadoss@gmail.com